Development process: how this model was built

Six generations of one fraud-screening model, from a variational autoencoder trained on two laptop cores to a hybrid that reads a customer's history, the entities around a payment and the fraud that has been confirmed so far - what each step changed, what it cost, what it did not do, and how to run the same thing yourself. The plain-language version covers the same ground with less vocabulary; and any term on this page explains itself when you rest the pointer on it (or tap it on a phone).

By Dean Mostafa. The code, the training pipeline and every pretrained generation on this site are in the repository, github.com/deanOcoin/fraudvae; this site is that repository running in its public mode, scoring only.

1. What this site is
2. The shape of the answer
3. Eleven public datasets, one schema
4. Generation by generation
5. What did not work
6. Labels take time
7. The numbers
8. Against the state of the art
9. The machines
10. Running it yourself
11. Testing it with your own transactions
12. What it is not
13. Credits and licences

1. What this site is

A demonstration of FinFraud, the fraud-screening model the site is named for. The **Score a transaction** tab takes one payment transaction - typed in, picked from the example rows, or uploaded as a CSV - and returns a probability of fraud with a receipt that says which fields drove it. The **Model version** control on that tab picks which of the six trained generations does the scoring, and **Compare every version** scores the same transaction on all of them at once, so the progress described below is something you can see on a single payment rather than take on trust. The **Model & training** tab shows each version's results and the timeline; **Training data** describes the training data; **How it works** is the technical walk-through.

What the site does not do: it does not train, it does not build data, it keeps nothing a visitor sends (no transaction is stored, no customer history is remembered between requests), and scoring is rate-limited per visitor so one person cannot occupy the server. The repository does all of those things - the training pipeline, the data build, the remote-training tooling - and the running it yourself section says what that takes.

2. The shape of the answer

Fraud comes in two kinds for a model. There is the fraud that looks like fraud already seen - the labelled history a bank has - and there is the fraud that looks like nothing seen before. A classifier trained on labels is the right tool for the first and blind to the second; a model of what *ordinary* traffic looks like catches the second and is weak on the first, because most fraud is ordinary-looking by design. The project kept both and let a small per-bank layer decide how much to trust each:

- **The novelty model** is a variational autoencoder (VAE) trained on legitimate transactions only. It learns to compress a transaction into a few numbers and reconstruct it; whatever it reconstructs badly is unusual, field by field. Since v5 it reads the customer's previous transactions first, so "unusual" means unusual *for this customer*. It never sees a fraud label, which is what makes it useful on fraud nobody has labelled yet.
- **The booster** is a gradient-boosted tree classifier (LightGBM) trained on the labelled fraud, over the same fields plus a block of context computed from the customer's and the merchant's past: bursts in the last hour and day, the customer's own amount z-score, tenure, first-time flags, and (v5) what has happened around the same counterparty, device, e-mail domain, address and card.
- **The blend** is one logistic regression over the calibrated signals with a weight per bank for each, fit on labelled validation rows. It is what the app flags on. Every bank also has a *generic* mode for a transaction from an institution the model never trained on.

- **v5's second opinions**: a per-bank specialist booster, a sequence head on the history read-out, a device head, an entity-graph network, text heads and behaviour verifiers - each a calibrated signal with a presence bit, used by the blend where it helps and ignored where it does not.

Every score comes with a receipt: the fields the novelty model could not explain and what it expected instead, the booster's per-feature pushes, which signals were present, which generation scored it and with how many of the fields it knows. The receipt is the part of the design that survived every generation unchanged.

3. Eleven public datasets, one schema

No single public dataset is both large and semantically rich, and a bank's own data looks like none of them exactly. So the corpus is eleven of them mapped onto one schema - 42.8 million transactions, 133 thousand labelled fraud (0.31%) - with each source keeping its own identity as a "bank" the models know by name. Two are real anonymised bank data (ULB/Worldline, IEEE-CIS/Vesta); the rest are simulations calibrated on real behaviour or synthetic sets published for research. The schema has 95 numeric fields and 24 categoricals that any source may or may not fill (an unfilled field is marked unobserved, never imputed), six entity keys that drive the customer sequence and the graph block but never enter a model, and per source an optional *wide block* of that source's own raw columns that only the booster reads. Splits are by time within each source: the oldest 70% trains, the next 10% calibrates, the newest 20% is the test set every number on this page comes from.

key	what it is	rows	fraud	licence	example rows here
fdh	Fraud Detection Handbook simulator: card payments at terminals, three fraud scenarios	1.75M	0.84%	no licence file on the data repository (the handbook's code is GPL-3.0)	no - regenerate with the handbook's notebook, or `fraudvae fetch fdh`
capone	Capital One synthetic card authorisations (recruiting dataset)	786k	1.58%	unspecified	no
tabformer	IBM TabFormer synthetic card transactions, 1991-2020	24.4M	0.12%	Apache-2.0	yes
ulb	ULB / Worldline real European card transactions, two days of 2013	285k	0.17%	DbCL v1.0	yes
paysim	PaySim mobile-money simulation	6.36M	0.13%	CC BY-SA 4.0	yes
sparkov	Sparkov simulated US card purchases with geography	1.85M	0.52%	CC0	yes
banksim	BankSim agent-based bank payments	595k	1.21%	not stated on the listing	no
ieee	IEEE-CIS / Vesta real card-not-present e-commerce (the Kaggle competition's training file)	590k	3.5%	competition rules, non-commercial	no
amlworld	IBM AMLworld HI-Small synthetic inter-bank transfers; the label is laundering	5.08M	0.10%	CDLA-Sharing-1.0	yes
baf	Bank Account Fraud (Feedzai, NeurIPS 2022): account applications with device telemetry	1.0M	1.10%	CC BY-NC-SA 4.0	yes
fraudecom	e-commerce first purchases with shared device ids	151k	9.4%	unspecified	no

None of the datasets is redistributed here. The six sources whose licences allow it ship a few hundred labelled example rows from the held-out test split (with attribution and the licence text beside them, in `data/examples/` of the repository); that is what the example picker on the Score tab draws from, and because they are test rows, the receipt can say whether the model was right. All eleven are public: the GitHub-hosted ones need nothing, the Kaggle ones a free Kaggle account (IEEE-CIS additionally means joining its competition and accepting the rules), and Kaggle refuses some VPN exit addresses. The **Training data** tab links to where each lives; `fraudvae fetch` in the repository downloads what it can (the GitHub-hosted ones directly, the Kaggle ones through the Kaggle API or a zip you drop in place) and then checks every file against the recorded size and checksum of the file the models were actually built from - the part that matters, because datasets get re-uploaded and re-cut, and a different edition builds fine and reproduces nothing. `fraudvae examples` then produces the same kind of example file from your own copy.

4. Generation by generation

The table is live - it is read from the runs present on this server - and the text underneath is the commentary. "Mean AUROC, 9 shared banks" is the like-for-like number across generations: the nine sources every generation since v4 was trained on. "Generic" is the score with the bank blanked, the number for an institution the model never saw.

Loading the timeline…

v3 on a laptop - `cpu_small-8src`

The first complete model: the VAE alone, 0.64 million parameters, trained on 3.5 million legitimate rows (eight sources capped at 1.5 million rows each) for ten epochs in 32 minutes on two CPU cores. No labels anywhere. Calibration picked the per-field normalised score over the raw one and froze per-bank thresholds on the validation split. Mean test AUROC 0.829: the simulators with structural fraud (BankSim, PaySim, Sparkov) and ULB's PCA space were easy for a density model; IEEE and TabFormer were middling; Capital One (0.52) and the handbook data (0.66) were the ordinary-looking fraud the design cannot see without labels.

v3 on the full corpus - `gpu_full-4090`

The same model, 3.4 times the parameters (2.19M), on every row: 25.6 million legitimate training rows, 23 epochs in 18.5 minutes on a rented RTX 4090 (21.7 minutes end to end, including scoring the 7.3-million-row test split). Mean AUROC 0.833 against 0.829. Seven times the data moved the mean by 0.004, and the validation curve was flat from epoch 9. That was the lesson of v3: the feature set, not the row count, was the ceiling. Whatever was left on the hard sources needed labels and context, not a wider decoder.

v4: labels, context and the blend - `gpu_full-4090-v4`

Three things, in order of how much they mattered. The **context block**: a 1-hour burst window, the customer's own amount z-score over 30 days, tenure, first-time flags (merchant, category, cross-border), the merchant's volume and amount-vs-merchant-mean - all causal, all computed at build time, because fraud lives in "a strange payment *for this customer*". The **booster**, trained on the labelled rows with legitimate traffic subsampled per bank and the sample weights capped so no bank defines "normal" for the others, early-stopped on the mean of the named-bank and the bank-blanked validation AUCs. The **blend**, replacing an earlier logistic head. AMLworld joined as a ninth source so the generic mode had seen more than one kind of institution.

The VAE itself did not move (0.826 - the context features changed nothing for the density model, for the third run in a row). The booster was the step change: mean blend AUROC 0.916, IEEE 0.75 to 0.86, TabFormer 0.91 to 0.98, Capital One 0.52 to 0.77, and recall at a 1% false-positive budget went from useless to usable on the real-ish sources (IEEE 0.1% to 35%, TabFormer 12% to 73%). Generic mode 0.74 to 0.956. The run took 16 epochs in 17.6 minutes on the 4090 plus an 8-minute post stage (booster, calibration, evaluation) - and the booster took three passes to get right, which is its own section below.

v5: history, entities, modules - `v5-h200-e`

The largest change. The VAE grew a **history encoder**, a small transformer that reads the customer's previous 32 transactions into a vector that conditions both the encoder and the decoder, so novelty is measured against this customer's own past. The schema grew the **entity-graph block**: for the counterparty, device, e-mail domain, address and card of a transaction, its 30-day volume, distinct customers seen with it, its age, and the fraud confirmed against it so far - counting only labels at least 30 days old, so a row never sees its own future. Each source gained a **wide**

block of its own raw columns for the booster alone (IEEE's 431 columns). Two sources joined (BAF, fraudecom), and five modules were trained on their own public data: per-bank specialist boosters, a sequence head, a device head, per-source GraphSAGE entity graphs, text heads and behaviour verifiers.

It ran on a rented H200 with the laptop off, through a relay: the raw files pushed once (18 GB), all eleven sources built in 7 minutes on 64 cores, the full corpus prepared in 95 seconds (30.0M train / 4.3M validation / 8.6M test rows, 589 features), then 30 epochs of the 3.01-million-parameter history model at 85 thousand rows a second - 5.8 minutes an epoch, 2.9 hours in all - followed by a 40-minute post stage for the booster (1,435 rounds in 285 s), eleven specialists, the heads, nine entity graphs, the text and behaviour modules, calibration and evaluation. Two earlier launches of the same configuration (c, d) had stopped on the wrong criterion: the per-epoch VAE AUROC, which swings by 0.3 on single sources because it scores with one posterior sample over small fraud sets. e selects on the held-out reconstruction objective itself, which moves smoothly.

On the nine sources v4 had, the blend went from 0.916 to 0.936, and the gains were where the new columns were: Capital One 0.765 to 0.838 (8% to 21% of its fraud at 1% FPR), IEEE 0.858 to 0.940 (35% to 56%). The ablation pinned it on the delayed-label entity features for Capital One and IEEE and the wide blocks for IEEE. The history-conditioned VAE became a better novelty model on the banks with customer sequences (Capital One 0.51 to 0.61, IEEE 0.73 to 0.79, AMLworld 0.82 to 0.91) and a worse one on the simulators (PaySim 0.94 to 0.80: a balance drained in one transfer is an outlier of the row, but the history model, having read the account's previous rows, half expects it). The blender gives the VAE score a weight near zero on most banks; its job is the receipt and the fraud nobody has labelled.

v5: one-week fraud labels - `v5-h200-f`

e left the handbook data at 0.69, where every model had left it, and the reason was mechanical. The simulator's second scenario compromises a terminal for 28 days; the schema's fraud-history features count labels at least 30 days old; by the time a terminal's fraud is "known", the terminal is clean again. The handbook's own baseline uses a one-week label delay for exactly these terminal-risk features. Whether a week is honest is a statement about a deployment's label latency, not about the model - so v5 carries both: the 30-day pair in the schema for the VAE and the booster, and the same pair at a 7-day delay in the wide block for the booster only, for every source with entities. Nothing about the VAE changed; this was a rebuild of the entity features and a refit of the booster side on a copy of e's checkpoint, 30 minutes on the box. fdh 0.691 to 0.893 (34% to 75% of its fraud at 1% FPR); IEEE 0.940 to 0.958; the nine-bank mean 0.936 to 0.961; generic 0.953 to 0.966.

v5: recent-fraud windows, latency as a setting - `v5-h200-h` (what ships)

Two more booster-side changes on the same checkpoint. **Recent-fraud windows**: the one-week fraud count and share over the entity's rows in the 7 and 30 days before the label horizon, so a terminal compromised this month is not diluted by a clean history. **Label-latency dropout**: a random 30% of the booster's training rows have the whole one-week block blanked, so a deployment whose labels take a month is a regime the model has seen rather than an undefined input; the score API takes `label_delay_days` (the "labels arrive within" control on the Score tab), and every generation since is evaluated twice, once per latency. 41 minutes of post stage. TabFormer 0.986 to 0.994 and 74% to 86% of its fraud at 1% FPR - card fraud on that file comes in bursts, and a lifetime share buried this month's burst under years of clean history; fdh 0.893 to 0.905. Scored as a deployment whose labels take a month, the same model lands within 0.001 of e on every bank. **0.963 mean over the nine shared banks, 0.928 over all eleven, 0.969 generic.**

5. What did not work

Documented because the negative results shaped the model as much as the positive ones. None of these is loadable on this site; the training notes have the full accounts.

- **More rows for the VAE.** Three times in a row (v3, v4, and v5's capped probes against the full runs) seven times the legitimate rows changed the density model by a few thousandths. The booster's quality was set by the features and the labels; its training set is 200 thousand legitimate rows per bank plus every fraud row whichever prepared set it is drawn from, so a laptop gets the same booster.
- **Per-field surprises in the blender.** The first v4 blender also took the 81 per-field surprise z-scores and the latent mean. Validation loved it (TabFormer 0.996); the test split hated it (0.839, below both models alone). The surprises are unbounded, the newest 20% of a 30-year file sits in a different regime (one column reached

z≈1000), and a slightly negative pooled weight let that one column outvote the booster. The blender now sees only bounded calibrated signals; the surprises stay on the receipt as explanation.

- **Un-capped sample weights.** Subsampled legitimate rows were weighted back to each bank's true base rate; on the full corpus that weight was 85 for TabFormer against 1 for ULB, so in the bank-blanked mode one bank's traffic *was* the definition of normal (generic 0.936 instead of 0.957 on identical rows). The weight is capped at 3 now.
- **Three times the legitimate rows per bank in the booster** (600k instead of 200k, with a higher learning rate) fit time-local patterns rather than fraud - a gender column became the third feature by gain - and did worse on the newest months than the 2-core baseline.
- **Lifetime entity counts.** The first v5 graph block carried "distinct customers ever seen with this entity" and "days since first seen". On a 30-year file those grow with the calendar; the booster read them as the era, the newest months were an era it had never seen, and TabFormer lost 0.05 AUROC and 57 points of recall. Windowed and capped versions replaced them (30-day counts, ages capped at a year): TabFormer back to 0.977, nothing else lost.
- **bf16 autocast with the history transformer.** The gradient norm overflowed around step 700 of the first epoch at any learning rate, with every individual gradient finite; a few steps later the embeddings were NaN. fp32 was 10% slower and clean, because the window gather on the CPU, not the matmuls, is the bottleneck. Inputs are bounded at ±20σ since, and a non-finite step is skipped rather than applied.
- **Stopping on the per-epoch VAE AUROC** (runs c and d): a one-sample estimate over small fraud sets that swung by 0.3 between neighbouring epochs. e stops on the held-out reconstruction objective.
- **Unregularised per-bank blend weights.** ULB has 21 fraud rows in the blender's fit half and the ULB interaction terms learnt them by heart (held-out 0.915 against the booster's 0.953). Hierarchical shrinkage - each bank's interaction columns scaled by √(fraud rows / 10,000) - pulled ULB onto the shared weights (0.953) and left the big banks alone.
- **The history encoder's read-out as booster input** (run i): a wash, 0.9627 against h's 0.9634 over the nine; the trees reached the same validation AUC in fewer rounds, so the 128 columns are not useless, but the velocity and entity blocks already say what the summary says. The option stays in the code as a documented negative result.
- **A longer VAE** (run g: 48-transaction window, 40 epochs) was stopped at epoch 7 once h's numbers were in: the blender gives the novelty score a weight of about zero, so a better novelty model would have changed the receipt and nothing the app flags on.
- **Plumbing.** LightGBM with unlimited threads on a 256-thread host ran five times slower than on two cores (it now caps itself at the cores it is allowed); a 24-million-row pandas datetime conversion blew a 6 GB build container twice; microsecond timestamps silently divided by the wrong constant until the velocity features looked absurd; a sorted memmap read returned rows in a different order than the labels it was paired with, which was the real cause of a generic mode at 0.50 that had been explained away as a modelling limitation.

6. Labels take time

A fraud label is not known when the transaction happens. A chargeback arrives weeks later; a customer's complaint, days; a terminal compromise is noticed when the pattern is. Any feature of the form "how much confirmed fraud has this entity had" is therefore a feature about the *past as it was known at the time*, and a model trained with labels it would not have had in production is lying about its accuracy. The schema's entity block assumes 30 days; the booster's one-week block assumes 7; the dropout trained the booster for both. On the Score tab, "labels arrive within" tells the model which deployment it is in: past a week, the one-week block is blanked before the booster sees it and the receipt says so ("used", "blanked", "not sent"). Every generation's results table shows both columns. The two numbers to read for the shipped model are 0.963 (nine-bank mean, labels within a week) and 0.935 (the same rows, labels within a month) - the second is exactly where the model stood before the one-week block existed, which is what honest degradation looks like.

7. The numbers

The shipped generation on the held-out test split (the newest 20% of every source, 8.57 million rows, 27,303 fraud; thresholds frozen on validation). **AUROC** is the probability that a random fraud row scores above a random legitimate one: 0.5 is a coin flip, 1.0 is perfect separation. **r@1%** is the share of the fraud caught when the alert budget is set to flag 1% of legitimate traffic, which is closer to how a fraud team experiences a model. The "30-day labels" column is the same model scored as a deployment whose labels take a month; the last two are the v4 release on the same rows.

source	VAE	booster	seq	local	device	graph	blend	r@1%	30-day labels: blend / r@1%	v4 blend	v4 r@1%
fdh	0.624	0.903	0.656	0.905	–	0.501	**0.905**	78%	0.679 / 34%	0.672	34%
capone	0.613	0.837	0.815	0.839	–	0.767	**0.841**	21%	0.836 / 19%	0.765	8%
tabformer	0.940	0.991	0.985	0.995	–	0.815	**0.994**	86%	0.986 / 73%	0.981	73%
ulb	0.962	0.986	0.987	0.985	–	–	**0.984**	89%	0.984 / 89%	0.982	84%
paysim	0.797	1.000	0.991	1.000	–	0.554	**1.000**	100%	1.000 / 100%	1.000	100%
sparkov	0.863	1.000	0.996	1.000	–	0.953	**1.000**	100%	1.000 / 100%	0.999	99%
banksim	0.990	0.999	0.999	1.000	–	0.998	**1.000**	99%	0.999 / 99%	0.999	97%
ieee	0.795	0.958	0.904	0.955	0.733	0.825	**0.958**	63%	0.940 / 56%	0.858	35%
amlworld	0.912	0.990	0.984	0.990	–	0.846	**0.990**	85%	0.990 / 85%	0.990	84%
baf	0.658	0.893	0.812	0.891	0.768	–	**0.893**	25%	0.893 / 25%	–	–
fraudecom	0.641	0.645	0.644	0.639	0.639	0.516	**0.641**	10%	0.641 / 9%	–	–
mean, 11 sources	0.799	0.927	0.889	0.927	0.714	0.753	**0.928**		0.904		
mean, v4's 9 sources	0.833	0.963					**0.963**		0.935	**0.916**	
generic (bank unknown)	0.625	0.969	0.930	–	0.735	–	**0.969**	70%	0.952 / 62%	0.956	66%

`seq` is the sequence head on the history read-out, `local` the bank's own specialist booster, `device` and `graph` the device-block and entity-graph heads where a source carries those columns. The modules trained on their own public data keep their own numbers: the URL head 0.994 AUROC, the free-text head 0.972, the profile head 0.783; the behaviour verifiers 0.11 (keystroke), 0.12 (motion), 0.17 (touch) and 0.24 (mouse) equal error rate; the graph encoder 0.78 / 0.88 / 0.92 / 0.98 on the DGraph-Fin / Elliptic / YelpChi / Amazon benchmarks.

How to read it: the simulators saturate because their fraud rules are learnable, which says nothing about real fraud. The real and real-ish sources are where the generations differ, and the hard ones stayed hard for honest reasons: fraudecom's fraud *is* a shared device and 95% of its test rows are a device seen for the first time; BAF is account applications with no customer history, and 55% of the fraud at 5% FPR is the range the dataset's own paper reports for gradient-boosted trees; Capital One's fraud is ordinary-looking purchases whose only strong public signal is a rare CVV mismatch.

8. Against the state of the art

Dataset by dataset, where a published number exists on comparable data, the shipped model is at or above it: ULB 0.984 against the 0.95-0.98 the gradient-boosting literature reports; the handbook data 0.905 against the handbook's own XGBoost baseline of about 0.87 with the same one-week assumption; BAF 55% of fraud at 5% FPR against "about half" in the dataset paper; IEEE 0.958 on a time split of the competition's training file, in the range of the competition leaderboard (which used a different split and a hidden test set, so this is not a ranking). On the simulators everything saturates and nothing can be concluded.

What a production system has that this does not, so the comparison stays honest: the features that separate ordinary-looking fraud (device fingerprints, consortium data across institutions, account-age and terminal signals the public columns do not carry); continuous retraining against label drift; sub-50-millisecond latency at thousands of transactions a second (this server scores one transaction in roughly 0.2 seconds on CPU, and the v3 VAE alone does 90 thousand rows a second in batch); case-management tooling, feedback loops and regulatory review. The architecture is a sound version of how the industry builds these systems - a supervised model on causal context features, a novelty model beside it, per-segment calibration, explanations on every score, label latency treated as a first-class input - and the realism of the data, not the modelling, is the gap. It is a demonstration, built in the open on public data, and its numbers are honest for that.

9. The machines

Every machine involved, what it did and what it was. Hardware is named because the question "would this be better on a bigger machine" has a precise answer (next section).

machine	what it did	hardware
the sandbox	Development and tests throughout; the v3 capped-corpus run (`cpu_small-8src`: 10 epochs, 32.5 min); the v4 capped baseline (`cpu_small-9src-v4`: 9 epochs, 33.6 min, then 25 min of booster, calibration and evaluation); the v5 development probes on two- and four-source capped sets.	A Linux cloud container: 2 x86-64 CPU cores, under 8 GB of RAM, no GPU. Python 3.13, PyTorch 2.14 (CPU).
the laptop	The app and the Compute tab that rented and drove the GPU boxes; the full raw-to-processed build of the corpus (2.6 GB of parquet); the inference timings quoted on this page (about 170 ms per scored transaction with the v5 model on the Apple GPU).	A MacBook Air with an Apple M3 chip and 24 GB of unified memory, macOS 26.6, PyTorch with MPS.
the 4090 box	The v3 full-corpus run (23 epochs, 18.5 min; 21.7 min end to end) and the v4 run (16 epochs, 17.6 min, plus 8.4 min of post stage), each provisioned from the laptop's Compute tab: 1.0-1.4 GB of processed parquet uploaded, `prepare` on the box in 71 s.	1× NVIDIA GeForce RTX 4090 (24 GB), 16 CPU cores allotted on a shared 256-thread host, rented by the hour (vast.ai). PyTorch 2.14.1+cu132.
the box (H200)	All of v5 with the laptop off, through the relay: the raw files pushed once (18 GB), the eleven-source build (7 min), the full `prepare` (95 s), run e (30 epochs, 171.8 min, then ~40 min of post stage), the f and h refits (29.8 and 40.6 min), the module training, the booster sweep and ablation, and the dead ends i and g.	1× NVIDIA H200 NVL (141 GB), 64 CPU cores, 188 GB of RAM, rented by the hour. PyTorch 2.14.1+cu130, fp32.
this server	The hosted demonstration: inference only, every generation loaded, rate-limited.	A VPS at 1984.hosting (Iceland): 6 vCPU, 16 GB of RAM, 320 GB SSD, no GPU.

10. Running it yourself

The repository trains everything on this page from the public downloads; nothing here needed access to anything private. Two facts about hardware first, because they decide what is worth doing:

A faster machine gives you the same model sooner, not a better one. The configuration that ships (`gpu_full`) is fixed: the same data, the same model size, the same epochs. On a slower GPU or a CPU it takes longer and arrives at the same place, up to the run-to-run noise of training. Where hardware did change the result was in *choosing a different configuration*: the `cpu_small` configuration is a smaller model (0.64M parameters against 2.2-3.0M) on a capped corpus (1.5M rows per source), chosen so a laptop finishes in half an hour. For v3 the two were indistinguishable (0.829 against 0.833); for v4 the capped CPU baseline landed 0.003 below the full run (0.913 against 0.916), because the booster's training set is the same size either way; for v5 the history encoder needs the long real sequences of the full corpus, so a capped CPU run of v5 should be expected to land clearly below the numbers above - it was used for development probes only and never trained to completion.

what	on a laptop (CPU / Apple GPU)	on one RTX 4090	on one H200 / H100
download the eleven raw datasets (`fraudvae fetch`)	roughly 15 GB of downloads (the AMLworld archive alone is 8 GB, of which one file is used), ~60 GB unpacked with the module datasets; minutes to hours depending on the connection, Kaggle credentials needed for eight of them		
build the corpus (`fraudvae build`)	1-3 hours (estimate; the full build ran on the laptop - the adapters stream the big files in chunks)	-	7 min on 64 cores
prepare the training arrays (`fraudvae prepare`)	10-20 min (estimate)	71 s (v4 schema)	95 s
v3 / v4 VAE, `cpu_small` (capped corpus)	~35 min	a few minutes	a few minutes
v4 VAE, `gpu_full` (full corpus)	~2 days (measured 9,800 rows/s on the laptop CPU: ~50 h for 30 epochs; early stopping ends it at about half)	17.6 min	under 10 min (estimate)
v5 VAE, `gpu_full` (history encoder, full corpus)	several days (estimate; the window gather is CPU-bound everywhere)	~1 day (estimate; 24 GB forces smaller batches)	2.9 h
the booster side (`fraudvae post`: booster, specialists, heads, graphs, calibration, evaluation)	1-3 hours (estimate; the booster itself is CPU work on 200k rows per bank and does not need a GPU)	~25 min	30-40 min

What the GPU time cost: the v3 and v4 runs on the 4090 were a few dollars together; the shipped v5 run (e plus the f and h refits) about $16 of H200 time; the whole v5 campaign, dead ends, module training and ablations included, about $105. The Compute tab in the repository's app benchmarks the real model on your machine and gives you this estimate before you rent anything.

You do not have to train. Every generation on this site is published as a pretrained bundle on the repository's Releases page, and `fraudvae models pull v5-h200-h` (or any other name from the timeline) drops it into `runs/`, where the app loads it. The v5 bundles omit one file: the PaySim entity graph (822 MB per run) whose head scored 0.55 on PaySim and carries no weight in the blend; the server falls back to scoring PaySim without it, as the receipt will say. In short:

```
git clone https://github.com/deanOcoin/fraudvae && cd fraudvae
python3 -m venv .venv && source .venv/bin/activate && pip install -e ".[dev]"
fraudvae models pull v5-h200-h          # the shipped generation, ~360 MB
fraudvae serve                          # http://127.0.0.1:8765 - the same app as this site,
with training enabled

# or train it yourself
fraudvae fetch                          # downloads what it can, tells you what needs Kaggle
credentials
fraudvae build && fraudvae prepare --max-rows-per-source 1500000
fraudvae train --config cpu_small      # half an hour on a laptop, the v3/v4-sized model
fraudvae train --config gpu_full       # the shipped configuration; see the table above for what
it takes
```

Requirements: Python 3.10+, PyTorch 2.1+, LightGBM 4+; for the full corpus, 60 GB of disk for raw plus processed data and 32 GB of RAM for the v4 prepare (64 GB or more for v5's); for inference alone, any machine with 4 GB of RAM free loads a v5 generation in a few seconds and scores a transaction in a fraction of a second. The README covers the macOS OpenMP detail that otherwise crashes the booster on the first call.

11. Testing it with your own transactions

Three ways, in increasing order of effort:

- **Example rows.** The Score tab's example picker draws labelled rows from the held-out test split of the six sources that may be redistributed. Pick a source, fraud or legitimate, and the receipt tells you what the model made of it and whether it was right - the label is shown next to the verdict. Any generation can score the same row.
- **A transaction you type in.** Fill in what you know - amount, hour, channel, whether it is the customer's first time at this merchant, the device, anything else the form offers - and leave the rest blank; unfilled fields are unobserved, not zero, and the model was trained to score with partial information. Choose the bank whose "normal" it should be judged against, or *generic* for a transaction from none of the eleven; generic is the model's answer for an institution it never saw. A customer history can be pasted as previous transactions so the v5 generations have a sequence to read.
- **Your own CSV.** The batch upload on the Score tab takes a CSV with any subset of the schema's columns (the column reference is on the Data tab) and scores up to a few thousand rows per upload here; the app on your own machine has no such cap. If the file has an `is_fraud` column the summary reports AUROC and recall on it, so you can evaluate a generation on data of yours without sending it anywhere - run the app locally for that.

What the server keeps of any of this: nothing. In public mode the customer store is read-only and empty, requests are not logged beyond nginx's access log, and the rate limit is per address.

12. What it is not

Not a product, and not a claim about real-world fraud rates. The corpus is public and mostly simulated; its labels are the datasets' labels; the real sources are anonymised to the point where many signals a bank would have are absent. A model trained on it scores a real transaction by analogy to the closest "bank" it knows, which is a useful demonstration of the method and not a fraud decision anyone should act on. The specific limits are catalogued in the **How it works** tab's last section and in the training notes: ordinary-looking fraud without terminal or device signals, first-seen devices, account applications with no history, and anything a 20-year-old simulated card file cannot know about 2026.

13. Credits and licences

The code is MIT-licensed. The datasets keep their own licences, listed above and in `docs/DATA_SOURCES.md`; the citation for each is in `fraudvae/data/registry.py`. The modules' training sets - malicious URLs, scam job postings, bot profiles, the Balabit, BB-MAS and HMOG behaviour sets, the DGraph-Fin, Elliptic, YelpChi and Amazon graph benchmarks - are likewise public and credited in the repository. The ideas are not new - heterogeneous-likelihood VAEs, gradient boosting on causal velocity features, per-segment logistic calibration, delayed-label features - and the training notes say where each came from; what is this project's is the particular combination, the eleven-way schema, the latency treatment and the receipts. Built in the open by Dean Mostafa; the repository's history is the record of how. A good part of the code was written with the help of an Anthropic Claude model - the front end more than the training side; the ideas, the design of the models and the data, and every decision about what to build and what to leave out are not the model's.